



Sistemas Distribuidos I (75.74)

Intro a Sistemas de Tiempo Real

Sistemas RT. Sistemas de Control.

Docentes

- Pablo D. Roca
- Gabriel Robles
- Franco Barreneche
- Nicolás Zulaica
- Franco Papa
- Manuel Reberendo
- Máximo Gismondi
- Camila Sebellin



- **Sistemas de Tiempo Real (RT)**

- Sistemas de Control



Sistemas de Tiempo Real (RT) | Definición

- Son aquellos sistemas cuya evolución se especifica en términos de requerimientos temporales requeridos por el entorno
- La correctitud del sistema depende de que entregue respuestas correctas y en tiempo correcto.
- Un sistema es RT si tiene al menos un servicio RT
- Ejemplos de sistemas RT:
 - Electrodomésticos digitales, medidores de señales (presión, pulsaciones, ritmo cardíaco, etc.) mediciones por sensores, control de automóviles, control en aeronaves, marcapasos, etc.



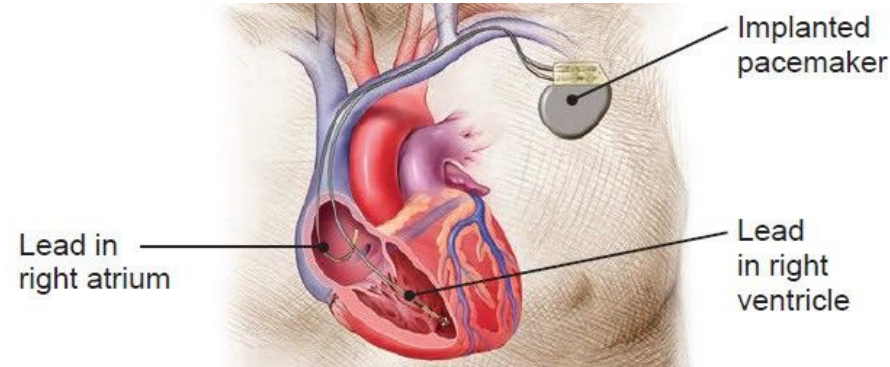
Hard RT:

- se debe evitar todo fallo relacionado con el tiempo de *delivery*
- perder un *deadline* o plazo de respuesta es un fallo total

Soft RT:

- fallos relacionados con el tiempo de *delivery* pueden ser admitidos ocasionalmente*.
- la utilidad de un resultado disminuye tras el *deadline*

* cierta frecuencia documentada





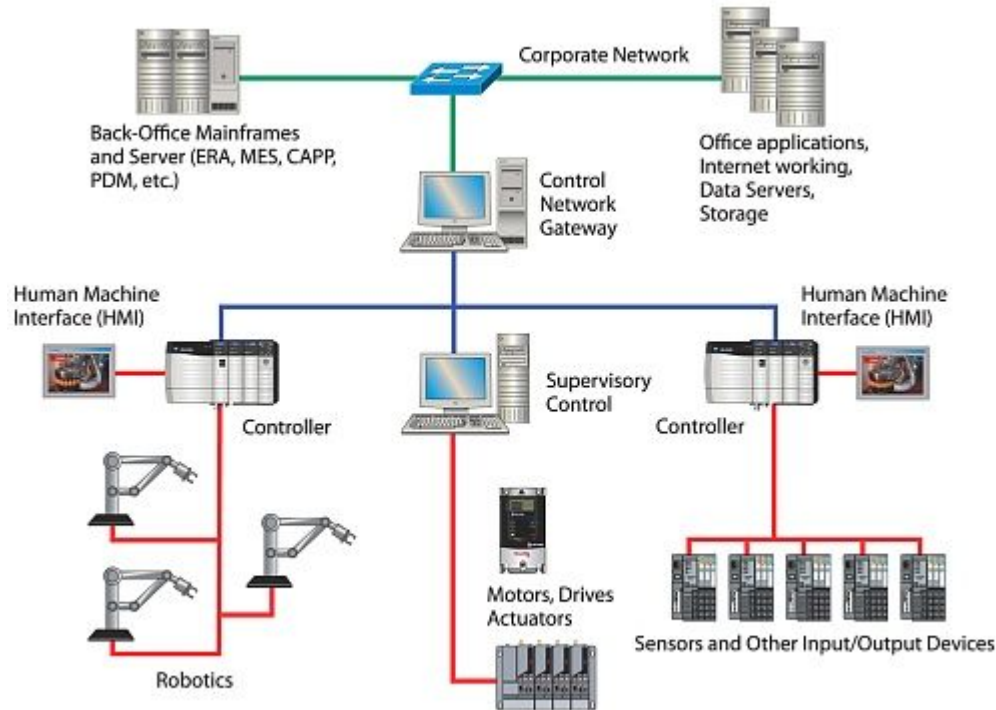
- RT implica previsibilidad, no performance.
 - Sistemas veloces pero sin previsibilidad
 - No son RT
 - Sistemas previsibles con tiempos característicos lentos
 - Son RT
- Se trata de hacer un correcto *scheduling* para que se cumplan los deadlines previstos por diseño



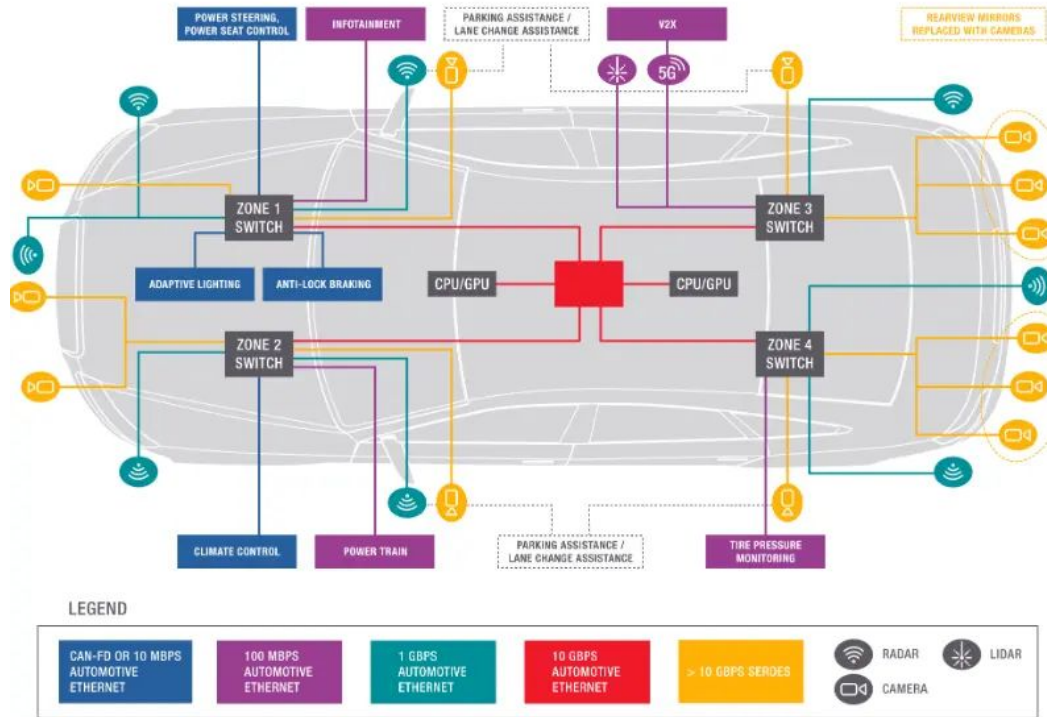
RT require comunicación fiable y sincrónica (con *deadlines* bien definidos)

- TCP/IP: no permite asegurar estos atributos.
- Comunicación Serial: permite control sobre estos aspectos
 - Ej. producto comercial: Profibus
- Ethernet: puede ser utilizado en capa física:
 - Evitar no-determinismos en protocolos capas superiores
 - *Switching* determinístico: necesario en colisiones en transmisión
 - Ej. producto comercial: Profinet

RT | Comunicación en automatización industrial



References:
<https://www.automationmag.com/ether-networking-how-ether-net-links-the-manufacturing-environment-with-the-enterprise-level-1940/>

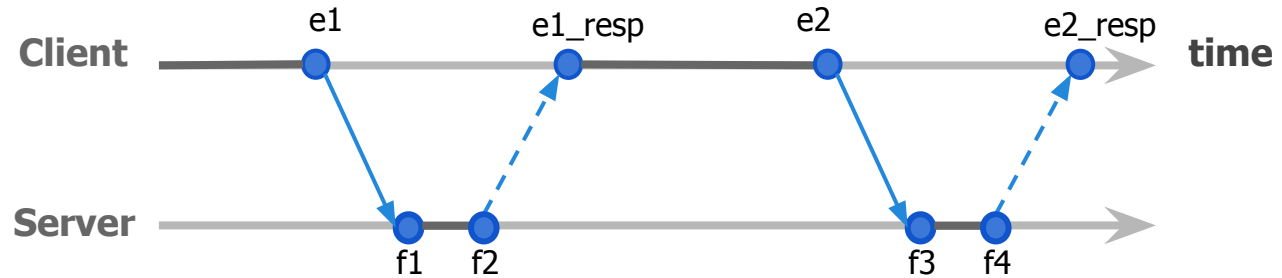




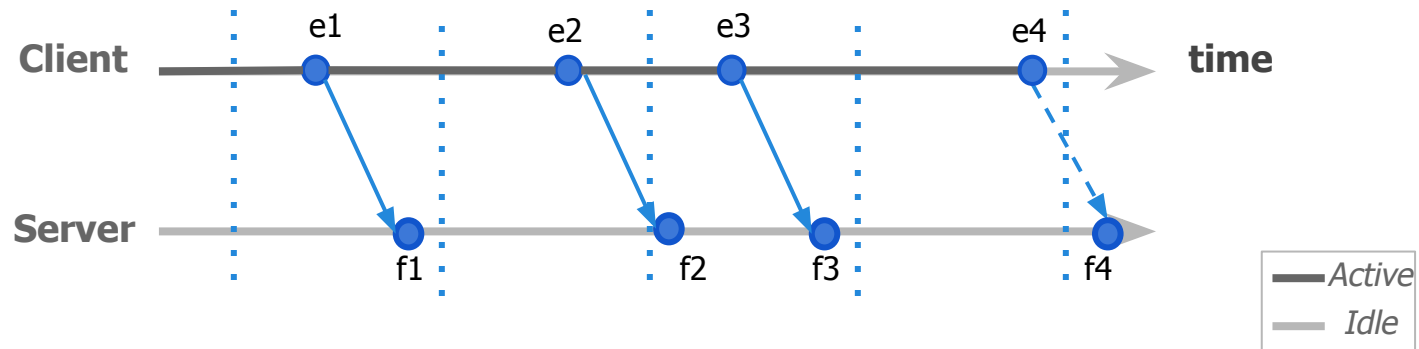
- Los sistemas ahora deben ser tolerantes a fallos de tiempo
- Distintos tipos de estrategias:
 - **Soft RT:**
 - Ej.: Sistemas web de gran escala
 - El 99% de los requests deben reponderse en 2 seg. El 1% restante se deben responder en 10 seg. Se admite 1 *outliers* cada 1M reqs.
 - **Hard RT:**
 - Ej.: Misión crítica
 - El 100% de los requests debe resolverse en 1 seg. Frente a errores, se asume un fallo catastrófico y se recomienda *hard reset*.
 - Muy importante revisar el factor de *Maintainability*.



Event - Triggered



Time - Triggered



Agenda



○ Sistemas de Tiempo Real (RT)

● **Sistemas de Control**



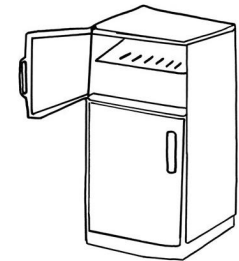
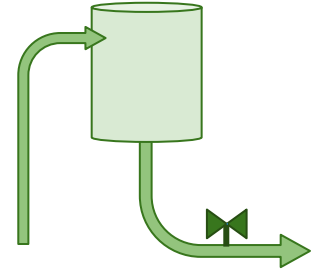
- Distintos escenarios cotidianos plantean un sistema a controlar de forma manual o automática

En la Industria

- Esquemas de irrigación
- Procesos de transferencia térmica
- Procesos químicos
- Líneas de producción

En la Vida Cotidiana

- Termostatos, ascensores, expendedores de líquidos, control de luminosidad, electrodomésticos en general, etc



**Control**

capacidad de actuar para garantizar el comportamiento de un suceso.

Variable controlada

cantidad o condición que se mide o controla.
Salida del sistema.

Perturbación

señal que tiende a afectar negativamente el valor de la salida del sistema.

Controlador (referencia)

sistema encargado de determinar la actuación para conseguir cierto objetivo del proceso

Proceso

toda sucesión de operaciones que se desea controlar.

Variable manipulada

cantidad o condición que se modifica para afectar el valor de la variable controlada.

Planta

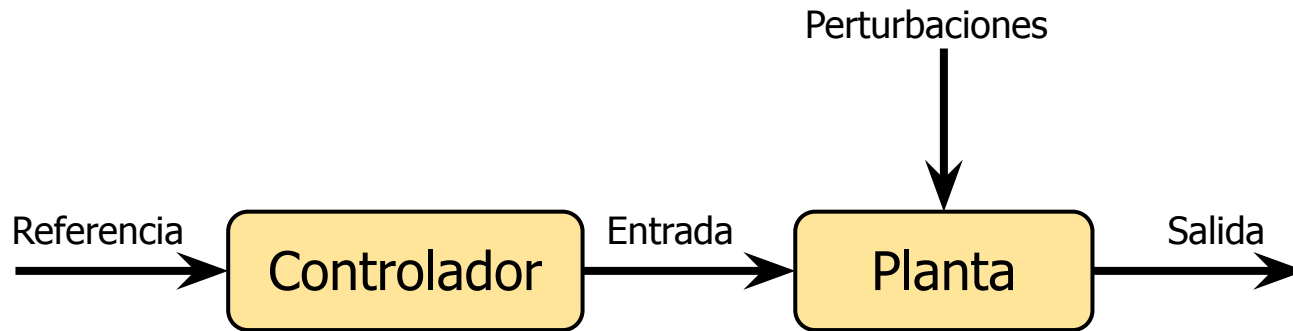
cualquier sistema físico que se desea controlar.

Actuador

elemento físico de la planta que frente a señales del controlador opera sobre el proceso

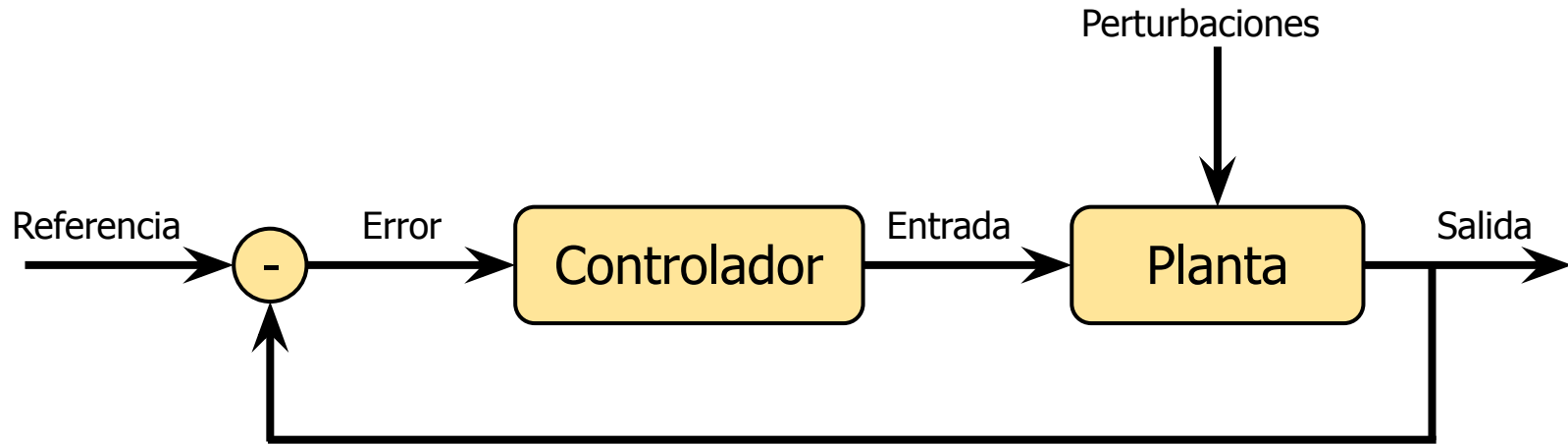


- **Control a lazo abierto:**
 - la salida del sistema no afecta la acción de control.





- **Control a lazo cerrado** o realimentado (feedback):
 - se utiliza información sobre el estado del sistema para actuar sobre el sistema y llevar la salida del mismo a los valores deseados.





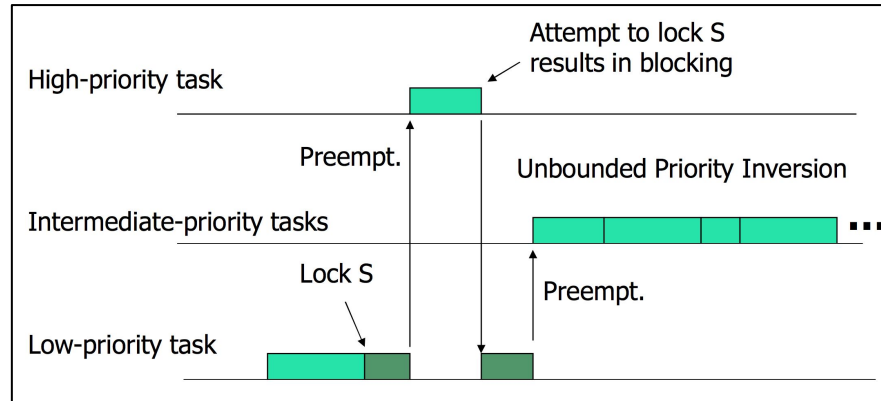
Programación y Tiempo Real

- Arquitecturas dirigidas por eventos (event-triggered) o por el tiempo (time-triggered)
- *Scheduling* es importante:
 - Apropiativo (non-*preemptive*)
 - Esquema de prioridades para poder cumplir *deadlines*
- Protocolos de comunicación específicos:
 - Determinismo en todas las capas
 - Evitar algoritmos de backoff => imprevisibilidad



Caso de Estudio: Mars Pathfinder

- Mars Pathfinder fue la primer misión a marte usando rovers. A los pocos días de aterrizar, se detectan continuos reinicios al intentar enviar datos.
- Motivos detectados:
 - **Watchdog** que reiniciaba el sistema al perderse el deadline de una tarea crítica.
 - **Inversión de prioridades:** una tarea de alta prioridad es interrumpida por una tarea de prioridad media.



Fuente: <http://blog.shiftefar.org/?p=207>



Caso de Estudio: Therac-25

- Therac-25 consiste de un acelerador de electrones y un sistema de control real-time para tratamientos de pacientes por radiación.
- Durante 2 años de empleo del equipo se detectaron al menos 6 fallos con pacientes recibiendo hasta 100x de la radiación recetada.
- Fallas presentes desde versiones anteriores del equipo (Therac-20) pero se manifestaron al eliminar controles por hardware y confiar en el software.
 - **Race Conditions:** UI mostrando modo erróneo al operador durante el bootstrapping del magnetrón
 - **Overflow:** 1 byte para contador de errores detectados. Overflow a 0 al detectar 256 chequeos fallidos...

PATIENT NAME: John		BEAM TYPE: E	ENERGY (KeV):	10
TREATMENT MODE: FIX				
	UNIT RATE/MINUTE	0.000000	PREScribed	0.000000
	MONITOR UNITS	200.000000		200.000000
	TIME (MIN)	0.270000		0.270000
GANTRY ROTATION (DEG)	0.000000	0.000000		VERIFIED
COLLIMATOR ROTATION (DEG)	359.200000	359.200000		VERIFIED
COLLIMATOR X (CM)	14.200000	14.200000		VERIFIED
COLLIMATOR Y (CM)	27.200000	27.200000		VERIFIED
WEDGE NUMBER	1.000000	1.000000		VERIFIED
ACCESSORY NUMBER	0.000000	0.000000		VERIFIED
DATE: 2012-04-16	SYSTEM: BEAM READY	OP.MODE: TREAT		AUTO
TIME: 11:48:58	TREAT: TREAT PAUSE	X-RAY		173777
OPR ID: 033-tfs3p	REASON: OPERATOR	COMMAND:		

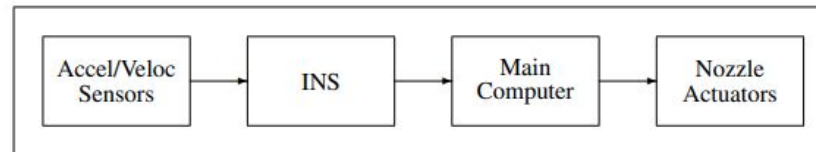
References:
<https://www.bugsnaq.com/blog/bug-day-race-condition-therac-25>

Leveson, Turner, An investigation of the Therac-25 accidents, IEEE Computer, 1997



Caso de Estudio: Ariane 5

- Ariane 5 fue un desarrollo de la Agencia Espacial Europea para colocar satélites en órbita.
- En su despegue de Junio/1996 el cohete modifica su trayectoria de forma brusca y estalla en el aire.
- Se hereda el sistema de navegación del Ariane 4, casi sin modificaciones (y casi sin testing)
- En un análisis post-mortem se detectan distintos fallos:
 - **Cast Overflow:** El algoritmo de navegación asume posibilidad de casteo de valores 64-bits a variables 16-bits sin chequear overflow.
 - **Error in Redundant Systems:** Los sistemas redundantes reprodujeron exactamente las mismas condiciones.





Caso de Estudio (Non RT): Mt. Gox

- Mt. Gox fue uno de los exchanges de Bitcoin más grandes del mundo.
- En 2011 un *glitch* en el software de generación de transacciones introduce un hash erróneo de la dirección del receptor.
- Las transacciones de Bitcoin pueden ser incobrables (*unredeemable*) por definición. No hay mecanismo para considerar detección de errores.
- Cerca de 2.500 bitcoins fueron aplicados en transacciones incobrables hasta que se detectó el error.

```
OP_DUP
OP_HASH160
f2e63314c350094550c703fcdcd
OP_EQUALVERIFY
OP_CHECKSIG
```

Expected

```
OP_DUP
OP_HASH160
0
OP_EQUALVERIFY
OP_CHECKSIG
```

Actual



Bibliografía

- P. Verissimo, L. Rodriguez: Distributed Systems for Systems Architects, Kluwer Academic Publishers, 2001.
 - Cap. 11 - Real Time System Foundations
 - Cap. 12 - Paradigms of Real Time
- Bob Martin, The Future of Programming, .NET Oxford, 2019.
 - <https://www.youtube.com/watch?v=BHnMItX2hEQ>